

Agent-Based Software Artifact Evaluation

Zhaonan Wu¹, Yanjie Zhao¹✉, Zhenpeng Chen², Zheng Wang³, and Haoyu Wang¹

¹Huazhong University of Science and Technology, Wuhan, China

{zhaonanwu, yanjie_zhao, haoyuwang}@hust.edu.cn

²Tsinghua University, Beijing China, zpchen@tsinghua.edu.cn

³Independent Researcher, hackerzheng666@gmail.com

Abstract—Artifact Evaluation (AE) has become a standard mechanism for linking software engineering papers to their supporting artifacts, but growing submission volume makes purely manual AE increasingly difficult to sustain. Although recent advances in LLM-based agents make AE automation increasingly plausible, current descriptive badge policies define badge semantics but provide no executable verification criteria, leaving both human reviewers and agents without a detailed basis for judgment. To address this gap, we construct ARTIFACTGUIDE, a structured scoring rubric grounded in ACM policy, expert-informed calibration, and artifact-based validation, and we design ARTIFACTCOPILOT, an agent collecting review evidence following a fixed sequence under ARTIFACTGUIDE and deriving the final badge decision from accumulated evidence.

We evaluate our framework on 60 real artifacts from recent software engineering conferences using human-adjudicated badges as reference. The results show that ARTIFACTGUIDE improves the AE performance of coding agents over official ACM badge-policy prompts, increasing three-run mean exact badge agreement by 10.55 to 28.34 percentage points. Across all evaluated systems and prompting protocols, ARTIFACTCOPILOT achieves the highest badge-level agreement at 70.56% and is the only system that completed all repeated runs successfully while producing a review report in every run. A controlled user study with 8 experienced researchers suggests that ARTIFACTCOPILOT reports improve reviewer confidence, help reviewers locate evidence, and understand evaluation scope more clearly. Further analysis translates insights from automated AE into practical guidance for designing higher-quality artifacts with clearer review routes, more explicit claim-to-output links, and more concrete reuse paths.

Index Terms—artifact evaluation, large language models, rubric design, LLM-based agent

I. INTRODUCTION

Since artifact evaluation (AE) was first deployed at EC/FSE in 2011 [21], it has been widely adopted across software engineering, programming languages, and broader computer science venues [40]. Nevertheless, neither its criteria nor its processes have kept pace with this proliferation. As submission and artifact volumes continue to grow, artifact review remains heavily reliant on manual effort, rendering the process increasingly unsustainable at scale [34], [9], [6], [14]. Recent advances in LLM-based software agents have substantially expanded the scope of automated software engineering tasks. Software agents can navigate repositories, execute commands, configure environments, and record execution traces [44],

[43], [39], offering a promising path toward automated AE. However, prior work has demonstrated that when LLMs or agents are deployed for evaluation tasks, performance depends critically on whether the rubric is sufficiently explicit and operational [28]. Therefore, automating AE with agents first requires a detailed rubric. In a similar vein, empirical findings in AE corroborate this requirement for a more detailed rubric. Hermann et al. [17] surveyed artifact evaluation committee members across major conferences and found that existing official policies fail to yield clear and consistent criteria for acceptable artifacts, while community expectations remain systematically unreflected in calls for artifacts, leaving reviewers without adequate guidance for badge decisions.

These observations point to a more fundamental issue: the AE community still lacks a concrete and operational rubric. As AE faces growing scalability challenges and agents are increasingly considered as a possible solution [18], [9], the need for such a rubric becomes even more pressing. We therefore argue that a concrete and operational rubric is an essential missing component of AE, for both human reviewers and agents.

Acting on this thesis, we first design ARTIFACTGUIDE, an operational artifact-evaluation rubric that bridges the gap between descriptive policy and badge decisions. ARTIFACTGUIDE translates badge semantics into an executable review standard through an availability gate, five assessment dimensions, and ordered intervention and evidence levels. We construct this rubric through three phases, deriving an initial version from official ACM policy and published AE guidelines, calibrating judgment boundaries through semi-structured interviews with 20 experienced artifact evaluators, and validating the revised rubric through manual reviews of real artifacts. On top of ARTIFACTGUIDE, we then design ARTIFACTCOPILOT, a specialized agent for rubric-guided artifact review that takes the paper and its corresponding artifact as input, uses phase-specific skills [7] to guide each review stage, produces structured evidence records, and derives final badge recommendations from them.

We then compared ARTIFACTCOPILOT against three coding agents and a project-execution agent under three review protocols with increasing structure, including official ACM badge policy [8], ARTIFACTGUIDE, and Skills+ARTIFACTGUIDE review. We also analyzed repeated-run stability and conducted a controlled user study with eight experienced researchers.

✉ Corresponding author: Yanjie Zhao (yanjie_zhao@hust.edu.cn).

The results show that ARTIFACTGUIDE improves the AE performance of coding agents over official ACM badge-policy prompts, increasing three-run mean exact badge agreement by 10.55 to 28.34 percentage points for OpenHands [39], Cline [13], and Codex [27]. Across all evaluated systems and protocols, ARTIFACTCOPILOT achieves the highest three-run mean exact badge agreement at 70.56%. It is also the only evaluated system that completed all repeated runs successfully and produced a review report in every run. The user study further suggests that ARTIFACTCOPILOT reports help reviewers locate evidence more effectively and recognize evaluation limitations more clearly.

Our further analysis shows that automated AE is useful not only for evaluating artifacts, but also for clarifying how higher-quality artifacts should be designed. Artifacts that can be reviewed stably by an agent tend to expose clearer review routes, more explicit claim-to-output links, and more concrete bounded reuse paths, whereas agent uncertainty often reflects underspecified artifact design.

Our main contributions are as follows.

- We design ARTIFACTGUIDE, a concrete and operational AE rubric that fills the gap between descriptive badging policy and actual badge decisions.
- We design ARTIFACTCOPILOT, a specialized agent for rubric-guided AE review, and show through systematic evaluation on 60 real artifacts that it achieves the highest badge-level agreement among all compared systems, reaching 70.56% exact badge agreement.
- We show that ARTIFACTGUIDE can be reused by coding agents as an effective external review structure, consistently improving their AE performance over official policy alone.
- We provide an in-depth analysis of failure modes in automated AE and derive practical implications for both reviewer-oriented AE automation and artifact design.

II. BACKGROUND AND MOTIVATION

A. Artifact Evaluation

The ACM artifact badging policy defines *Available*, *Functional*, and *Reusable* badges as a shared institutional framework for artifact quality [8]. Since its pilot at ESEC/FSE 2011 [21], AE has become standard practice across major software engineering venues [4], [2], [3], [5]. Yet the institutionalization of AE has exposed a tension between badging policy and review practice. Existing criteria remain descriptive rather than operational, specifying what each badge represents without defining how reviewers should verify it. This gap leaves individual reviewers to reconstruct judgment standards from personal experience, which undermines consistency and complicates scaling. Even when artifacts execute successfully, reviewers face unsupported decisions about whether observed deviations still permit reasonable claim support [14]. The resulting subjectivity not only threatens evaluation reliability but also drives resource demands. Each Artifact Evaluation Committee (AEC) member is typically assigned only 1–3 artifacts [14], and EuroSys 2025 required 98 committee

members to sustain this practice. As submission volume grows, this labor-intensive model is becoming unsustainable [34], [9].

B. LLM-based Agents

The continuing improvement of LLM capabilities has driven the formation of a new automation paradigm, namely agent systems centered on LLMs [37], [41], [42]. LLM-based agents extend single-shot inference by accumulating context across steps, invoking external tools, and incorporating execution feedback into the reasoning process [37], [44], which has been applied in SE to code repair, vulnerability detection, and test generation [10], [32], [24], consistently demonstrating advantages on tasks requiring cross-step reasoning and environmental feedback [43], [39]. Critically, agent execution trajectories constitute traceable records of task states [29], a property that recent work has begun to exploit for evaluation itself, replacing static assessment with executable dynamic processes [46], [16]. This traceability directly addresses AE’s core need for evidence-based judgment.

C. Problem Formulation

These observations suggest that AE automation requires both explicit standards and executable enforcement. We define automated AE as an agent-supported, rubric-based evidence review task. The rubric provides the what, decomposing badge standards into explicit inspection items, evidence requirements, and decision thresholds. The agent provides the how, executing these items as bounded environment interactions and accumulating structured evidence. Neither alone suffices. A rubric without execution remains static inference. An agent without rubric guidance operates without evaluation criteria. Together, they shift AE from subjective badge assignment to traceable, auditable evidence review.

III. THE DESIGN OF ARTIFACTGUIDE

A. Scope and Design Principles

We design ARTIFACTGUIDE as a multi-level review framework that operationalizes descriptive badge standards. By specifying the concrete items to be checked, the evidence to be recorded, and the logic by which evidence supports conclusions, it provides a unified basis for artifact evaluation without redefining ACM badges or superseding venue-specific policies. ARTIFACTGUIDE is designed around three interlocking principles. First, **policy alignment** requires that the scope and criteria of ARTIFACTGUIDE remain faithful to the official definitions of the badges. Second, **analytic decomposition** requires that ARTIFACTGUIDE decomposes badge judgment into separately checkable intermediate dimensions. This design draws on the established distinction in scoring rubric design between analytic and holistic rubrics [19]. Third, **evidence grounding** requires that each dimensional judgment be tied to concrete objective evidence, making each determination traceable.

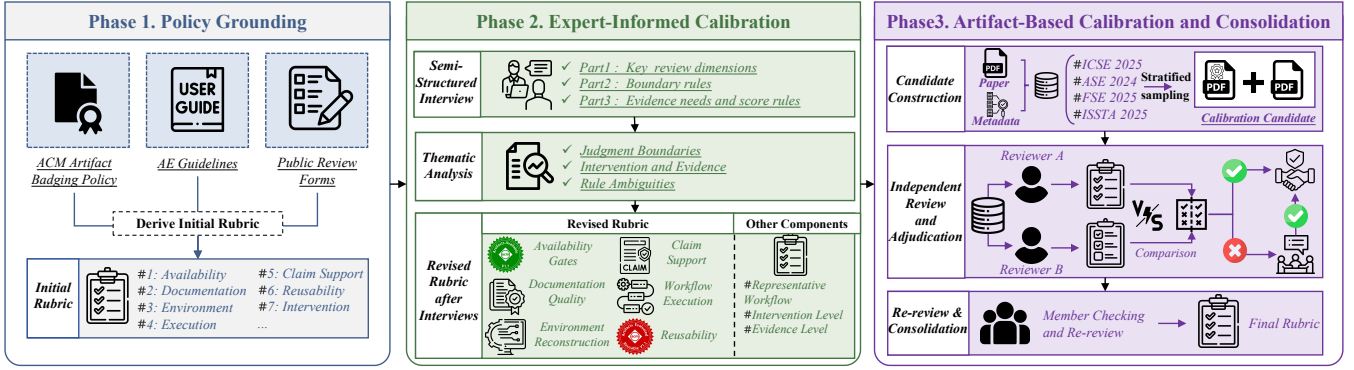


Fig. 1: The Workflow of the Construction of ARTIFACTGUIDE.

B. Construction of ARTIFACTGUIDE

Based on the established methodology for rubric construction proposed by [20], [30], ARTIFACTGUIDE was constructed through a three-phase process. We first conducted policy grounding to establish the rubric’s policy basis and then revised the draft through expert calibration. Finally, we consolidated the rubric through manual review of real public artifacts. Figure 1 illustrates this construction process.

Phase 1. Policy Grounding. We built the initial version of ARTIFACTGUIDE based on the ACM artifact badging policy [8], combined with specific requirements from recent AE guidelines of ICSE, ASE, FSE, and ISSTA. We further consulted publicly available review forms and guidelines to capture signals that are commonly omitted in formal badge descriptions but regularly attended to in actual review practice, such as whether the artifact is obtainable and has a clear entry point, and whether installation and execution instructions are complete. Synthesizing these sources, the initial version established the basic framework for subsequent iterations, covering availability, documentation, environment, execution, claim support, reusability, and reviewers’ interventions.

Phase 2. Expert-Informed Calibration. While the initial version captured review objects that commonly arise in AE, it fell short of reflecting what real reviewers actually prioritize in practice. We therefore revised it through expert-informed calibration, interviewing twenty participants with artifact-related experience. The participant pool covered three forms of experience relevant to AE: prior service on artifact evaluation committees, paper submission and artifact preparation, and substantial hands-on experience in reproducing published research artifacts. These categories were not mutually exclusive, and several participants had experience across multiple roles. This composition allowed the calibration to reflect reviewer-side judgment, author-side preparation, and practical reproduction work. The final five interviews yielded no new dimensions, suggesting that the core themes had stabilized, so we concluded recruitment.

Interviews were conducted as one-on-one semi-structured sessions. Participants were first asked to describe key review

dimensions based on their own experience. Then the initial version was presented, and boundary rules were discussed, such as the acceptable scope of reviewer interventions and the distinction between execution success and result support. After the interviews, we conducted thematic analysis [12] of the interview records. Two authors independently completed initial coding and resolved coding differences through discussion. A feedback item was treated as a generalizable rule only when it recurred across multiple participants’ descriptions, rather than being tied to a particular artifact or an individual participant’s preference. Compared with the initial version, the revised ARTIFACTGUIDE established clearer judgment boundaries across the Availability gate and the five dimensions of documentation quality, environment reconstruction, workflow execution, claim support, and reusability. Furthermore, we summarized the concepts of representative workflow, evidence levels, and intervention levels.

Phase 3. Artifact-Based Calibration and Consolidation. Phase 2 calibrated key judgment dimensions and evidence requirements through interviews, but the feedback remained limited to participants’ discussions of prior experience and hypothetical scenarios. It was insufficient to demonstrate that the revised rubric could be applied consistently in real artifacts. We therefore further validated the revised rubric through manual review of real public artifacts. We constructed a calibration set from 642 research track papers across ICSE 2025, ASE 2024, FSE 2025, and ISSTA 2025, selecting those with publicly accessible artifacts and stratifying by official badge status. We used ASE 2024 rather than ASE 2025 because ASE 2025 did not include an artifact evaluation track.

We sampled 16 badged and 8 unbadged artifacts, a 2:1 ratio intended to test the rubric against both categories. Two authors independently reviewed the 24 artifacts using the revised rubric, achieving 83.3% agreement on the availability gate, 74.2% on 120 dimension-level judgments, and 54.2% on final badge types. The badge-level disagreements exposed four boundary ambiguities, including whether the availability gate checks completeness or mere accessibility, whether Functional requires full reproduction or accepts representative execution,

TABLE I: Gate conditions and dimension-level scoring criteria of ARTIFACTGUIDE. Intervention levels (I0–I4) and evidence levels (E0–E3) are defined in Table II.

Component	Scoring Criteria
Availability gate	GC1: publicly accessible and downloadable; GC2: README or other entry exists; GC3: related to the paper and contains material beyond the text.
Documentation Quality	R: no usable entry, no environment or workflow instructions; WR: entry exists, but key information is largely missing; WA: goal, setup, workflow, I/O, and paper mapping documented with minor inference gaps; A: core guidance plus structure, dependencies, resources, and failure boundaries.
Environment Reconstruction	R: no usable path, failed setup, or depends on I4-level intervention; WR: setup described but requires substantial repair; WA: reaches evaluable state with I0–I2 or semantic-preserving I3; A: stable, isolated, repeatable setup with clear dependency information.
Workflow Executability	R: no workflow or no checkable execution state; WR: candidate fails, lacks valid output, or only runs trivial commands; WA: produces E2/E3 valid output without I4 intervention; A: completes stably with I0 or I1 and checkable output.
Claim support	R: no E2/E3 evidence or no explainable paper mapping; WR: output maps weakly or peripherally to paper; WA: maps to at least one method, claim, result, table, figure, or experiment; A: supports the main result or full reproduction with a traceable path.
Reusability	R: no functional base, or no reuse entry identifiable; WR: reuse entry exists, but transfer information is insufficient; WA: transfer validation succeeds on new input, configuration, or scenario; A: reuse path clear, low-intervention, repeatable.

TABLE II: Representative-workflow checklist, intervention and evidence levels, and badge-derivation rules of ARTIFACTGUIDE.

Component	Criteria
Representative workflow	RW1: grounded in README, docs, paper appendix, author script, or official workflow; RW2: invokes an evaluation-relevant program, script, notebook, service, demo, benchmark, analysis, test, or replication path; RW3: produces interpretable valid output; RW4: maps output to a paper method, result, claim, table, figure, or artifact target; RW5: states coverage limit if not full reproduction.
Intervention level	I0: run as documented; I1: install dependencies, set environment variables, or choose compatible versions; I2: fix paths, commands, startup order, or wait time; I3: semantic-preserving configuration, wrapper, non-core script, or compatibility change; I4: semantic change to algorithm, model, metric, experiment, result generation, or core data processing.
Evidence level	E0: no executable evidence; E1: static or precomputed materials only; E2: reviewer runs representative workflow and obtains valid output; E3: reviewer runs full or near-full main workflow and obtains paper-facing result objects.
Badge rules	Available: GC1–GC3 pass; Functional: Available, D2–D4 $\geq$ WA, E2 or above, no I4; Reusable: Functional, both D1 and D5 $\geq$ WA, transfer validation succeeds, no I4.

whether Reusable demands independent input construction or permits provided demo scripts, and whether external dependencies block scoring or are recorded as limitations. A third author adjudicated these cases to resolve scores and identify root causes. After all calibration artifacts were fully evaluated and adjudicated, ARTIFACTGUIDE was finalized. Revisions clarified the evidence-to-claim mapping, delineated the boundary between workflow execution and result support, and made reviewer action recording explicit to prevent conflating original artifact quality with post-intervention success. The two reviewers then re-reviewed all 24 calibration artifacts, eliminating residual disagreements caused by rule ambiguity and reaching full agreement. The finalized version was submitted to the interview participants, and none raised objections to the final design of ARTIFACTGUIDE.

C. Structure of ARTIFACTGUIDE

The finalized ARTIFACTGUIDE first applies an Availability gate to determine whether an artifact satisfies the minimum material conditions for substantive review. It then evaluates five core dimensions, D1 through D5, covering documentation, environment reconstruction, workflow execution, claim support, and reusability. Each dimension follows a four-level scale of reject (R), weak reject (WR), weak accept (WA), and accept (A), adopted from traditional academic peer review [36], with WA serving as the minimum positive threshold for satisfying the corresponding badge condition. The five dimensions are

grouped by badge tier. D2 (environment), D3 (execution), and D4 (claim support) determine whether an artifact reaches the Functional threshold, while D1 (documentation) and D5 (reusability) are additional requirements for the Reusable badge. This grouping reflects the empirical finding that an artifact must first be executable before its reusability can be assessed, and that documentation quality is not necessary for functional correctness. To prevent any runnable command from being treated as valid review evidence, ARTIFACTGUIDE also defines representative workflow conditions (RW1 to RW5), evidence levels (E0 to E3), and intervention levels (I0 to I4). The intervention levels distinguish original artifact quality from reviewer-assisted success, ensuring that post-intervention execution does not inflate the badge recommendation. The evidence levels ensure that only reviewer-executed outputs (E2 or above) count toward badge decisions, excluding static materials or precomputed results. Table I summarizes the availability gate and dimension scoring criteria, while Table II presents the supporting criteria for determining evidence validity and final badge assignment. Due to space constraints, the complete ARTIFACTGUIDE with illustrative examples is available in our open-source artifact [1].

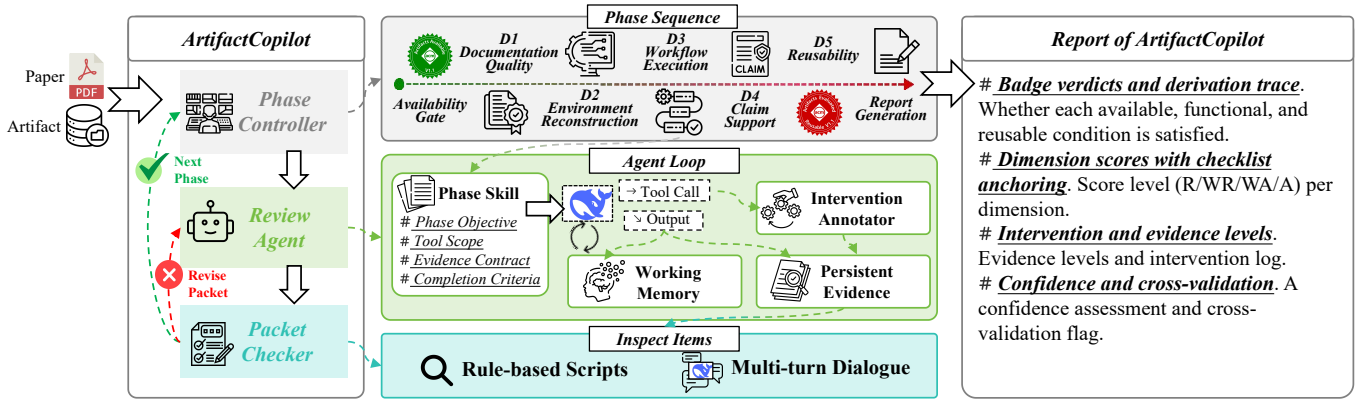


Fig. 2: Overview of ARTIFACTCOPILOT.

IV. ARTIFACTCOPILOT

A. System Overview

The ARTIFACTCOPILOT module models AE as a phased evidence review process. ARTIFACTCOPILOT first checks artifacts in a fixed phase sequence, generating verifiable evidence packages at each phase. The system only proceeds to the next phase after the phase evidence passes the check. The final badge is derived from persistently accumulated evidence rather than from the agent’s immediate response. As shown in Figure 2, the input to ARTIFACTCOPILOT is the paper and its corresponding artifacts. The **Phase Controller** and **Packet Checker** are responsible for the phase progression and acceptance. The execution of each phase is handled by the **Review Agent**, which invokes tools under the constraints of the current phase skill, observes the output, and records the execution evidence and intervention actions during the process. Upon completion of the final phase, ARTIFACTCOPILOT outputs a structured review report.

B. Phase Orchestration

The control layer of the ARTIFACTCOPILOT module is responsible for transforming ARTIFACTGUIDE from static scoring criteria into an executable review process. The Phase Controller enforces a fixed phase sequence, starting with the Available Gate. Within each phase, the Review Agent can choose specific tool calls and inspection paths based on the specific characteristics of the current artifact; however, the phase objectives, permitted operations, evidence required for phase completion, and completion conditions are all defined by the phase skill. Therefore, ARTIFACTCOPILOT can adapt to different artifacts while avoiding the invention of review methods in each evaluation.

The Packet Checker verifies the evidence packet before phase transition. At the end of each phase, the Review Agent must submit a packet containing the required evidence artifacts and phase conclusions. The checker applies two lightweight checks. First, it runs rule-based validation to confirm that the required files exist and that the packet is structurally complete.

Second, it invokes a bounded dialogue verifier with a maximum of five turns to examine whether the stated phase conclusions are actually supported by the submitted logs and related documents. This second step is intentionally lightweight whose role is only to check evidence-conclusion consistency, not to re-review the artifact or produce new judgments. Only packets that pass both checks are committed and allowed to advance to the next phase. Otherwise, ARTIFACTCOPILOT returns to the current phase to supplement or revise the evidence.

It is important to emphasize that the Availability gate incorporates a unique early stopping branch. If an artifact fails the checks of GC1–GC3, the Phase Controller will automatically skip directly to the phase of report generation, avoiding excessive resource consumption on artifacts that do not meet the minimum evaluation criteria.

C. Review Agent

Skill-scoped execution. The Review Agent is responsible for evidence collection and integration into report generation. Each phase is governed by a phase skill, a pre-written instruction artifact that specifies four elements: Phase Objective, Tool Scope, Evidence Contract, and Completion Criteria. Upon entering a new phase, the agent reads the phase skill and invokes tools within its Tool Scope to produce evidence that satisfies the Evidence Contract and Completion Criteria. These skills limit the agent’s operational space. For example, the document quality phase primarily allows reading and observing the document structure, while the environment reconstruction phase allows dependency probing and environment configuration but strictly prohibits the execution of representative workflows. This progressive scoping prevents evidence contamination between phases.

Dual memory and persistent evidence. The Review Agent maintains two types of memory to prevent temporary planning and partial observations from being conflated with auditable evidence. The first is working memory, which stores the current phase’s transient state, including current tasks, recent observations, and pending execution plans. The second is

persistent memory, which stores the phase evidence that is eligible for submission, including phase summaries, instruction logs, and intervention records. This information is permanently saved upon phase completion and serves as the direct source of the final report.

Intervention annotation. The Review Agent records actions that introduce reviewer-side assistance beyond the artifact’s original instructions, such as installing missing dependencies, modifying paths or startup order, supplying external inputs, or applying compatibility-oriented fixes. Each such action is checked by a script-based intervention rule and logged with its purpose, intervention type, and anticipated downstream impact. The resulting record becomes part of persistent evidence, preventing the agent from retrospectively downplaying the impact of an intervention after observing successful execution.

D. Structured Review Report

The final phase produces a structured review report in both Markdown and JSON formats, which are structured to support human audit, deriving all claims from persistent evidence.

Badge verdicts and derivation trace. The first component details the badge verdicts and their derivation traces. For each Available, Functional, and Reusable badge, the report explicitly states the deterministic rule applied and links directly to the specific evidence artifacts that satisfy the gate conditions. For instance, a Functional badge decision cites the workflow execution log from dimension D3 and the claim mapping file from dimension D4. All references include hyperlinks to the corresponding run directories for immediate verification.

Dimension scores with checklist anchoring. The second component presents the dimension scores with checklist anchoring. For every score from dimension D1 to D5, the report enumerates the specific checklist items from ARTIFACTGUIDE that were satisfied or violated. Each item is directly linked to the corresponding command output or log excerpt. This granular linkage empowers reviewers to inspect the foundational evidence and substitute their own judgments where necessary.

Intervention and evidence levels. The third component is a consolidated ledger of intervention and evidence levels. It records all interventions from level I0 to I4 alongside their rubric justifications and assigns the appropriate evidence level from E0 to E3 for each dimension. Such transparency is crucial for badge decisions because the rubric disqualifies level I4 interventions from receiving Functional or Reusable badges.

Confidence and cross-validation. The fourth component provides reviewer-facing confidence cues and consistency flags. These signals do not directly affect the final badge decision, which is still derived from the deterministic rules of ARTIFACTGUIDE. Instead, they help reviewers identify cases that may require closer inspection. Confidence is estimated from the completeness of supporting evidence and the intervention burden recorded in the run. Cross-validation applies a small set of rule-based consistency checks, for example whether $D3 \geq WA$ is accompanied by E2 or above, whether $D4 \geq WA$ is supported by a successful execution result, and

whether a Reusable recommendation is compatible with the Functional gate and D5 judgment. The report lists any resulting flags according to their severity.

V. EVALUATION

We organize our evaluation around four research questions.

- **RQ1.** How closely does ARTIFACTCOPILLOT align with human adjudication?
- **RQ2.** How does ARTIFACTCOPILLOT compare with baseline agents under different review guidance?
- **RQ3.** How stable is ARTIFACTCOPILLOT’s behavior across repeated runs?
- **RQ4.** How useful are ARTIFACTCOPILLOT’s reports in AE review?

A. Experimental Setup

a) *Datasets:* Prior work has reported that a portion of badged artifacts become inaccessible or non-functional over time [33], [25]. We therefore chose to re-execute the evaluation process rather than rely solely on previously awarded badges as ground truth. From the 642-paper pool in § III, we excluded calibration samples and artifacts with broken links, yielding 369 artifacts. To cover the judgment boundaries of AE while keeping manual review costs manageable, we performed a pre-labeling pass based on manual inspection without full evaluation. We stratified the pool into 104 cases that likely reach the Functional threshold, 246 boundary cases whose status cannot be determined by initial inspection, and 19 negative cases that clearly fail the Availability gate. Then we drew a stratified sample of 60 artifacts, 30 predicted to at least Functional, 20 boundary samples, and 10 predicted to fail Available. Two authors independently evaluated all 60 artifacts using the finalized ARTIFACTGUIDE and assigned dimension scores, gate decisions, and final badge recommendations. Disagreements were then adjudicated through discussion with a third author based on the recorded review evidence. The initial independent badge-level judgments achieved a Cohen’s kappa of 0.85 [35], indicating strong inter-rater agreement. The final distribution consisted of 6 *no-badge*, 20 *Available-only*, 13 *Available+Functional*, and 21 *Available+Functional+Reusable*. Under the Functional binary threshold, 34 artifacts were positive, and 26 were negative.

b) *Baselines:* We compare ARTIFACTCOPILLOT against three representative coding agents and a project-execution agent. OpenHands [39] represents open-source general-purpose software engineering agents with strong repository navigation and development capabilities. Cline [13] represents interactive coding assistants optimized for file-level editing and terminal use. Codex [27] represents leading commercial coding agents with strong code understanding and generation capabilities. For Codex, we use a non-interactive configuration so that the full evaluation pipeline runs without human intervention. ExecutionAgent [11] represents project-execution agents designed to make repositories runnable and testable. For all baselines, we keep the agent framework unchanged,

TABLE III: RQ1: Alignment with human adjudication across badge, gate, and dimension levels.

Layer	Item	Description	Acc.	Prec.	Rec.	F1
Badge	Exact match	Four-level badge	70.56%	74.05%	72.05%	70.64%
Gate	Available	GC1–GC3 pass	–	98.14%	97.53%	97.82%
	Functional	$\geq$ Available + D2–D4 $\geq$ WA	–	98.55%	70.59%	82.23%
	Reusable	$\geq$ Functional + D1,D5 $\geq$ WA	–	87.44%	49.21%	62.83%
Dimension	D1	Documentation quality	93.33%	95.30%	96.60%	95.95%
	D2	Environment reconstruction	83.33%	86.43%	91.67%	88.97%
	D3	Workflow executability	80.56%	86.46%	79.05%	82.59%
	D4	Claim support	80.56%	92.22%	74.77%	82.59%
	D5	Reusability	77.78%	78.26%	54.55%	64.29%
	<i>Mean over D1–D5</i>		83.11%	87.73%	79.33%	82.88%

inject the corresponding review protocol as task guidance, and enlarge step budgets to reduce premature termination.

c) *Metrics*: We evaluate the system at three levels. At the badge level, we report exact-match accuracy and macro-averaged precision, recall, and F1 across the four badge classes. At the gate level, we treat Available, Functional, and Reusable as nested binary decisions and report precision, recall, and F1 for each. At the dimension level, we report four-class accuracy on the R/WR/WA/A scale. We also report a binarized version of the dimension-level results, collapsing R/WR into negative and WA/A into positive, and computing the corresponding precision, recall, and F1. Finally, we use the Functional threshold as an artifact-level binary separator, treating artifacts that meet it as positive and those that do not as negative, to assess whether the system can distinguish functionally valid artifacts from invalid ones.

d) *Implementation Details*: Due to cost considerations, we selected DeepSeek-V4 Flash as the underlying language model for all systems. For Codex, which natively operates through OpenAI infrastructure, we accessed DeepSeek-V4 Flash via CC-Switch [15] to align the model backend as closely as possible across agents. For other baselines, we used their default configurations with enlarged step budgets to prevent premature termination. OpenHands was configured with 150 max iterations and 10 retries, Cline with 150 max requests and no timeout, and ExecutionAgent with 150 max steps and 1 retry. ARTIFACTCOPILOT was configured with 150 max steps. Experiments were conducted on a machine equipped with NVIDIA A100-80G GPUs running Ubuntu 22.04. All results reported for RQ1 and RQ2 are three-run means. RQ3 instead reports stability statistics aggregated over the three repeated runs.

B. RQ1: Agreement with Human Adjudication

ARTIFACTCOPILOT achieves 70.56% exact agreement with human adjudication at the badge level. Agreement is strongest for judgments grounded in directly observable evidence. The Availability gate attains 97.53% recall, and D1 records the highest four-class accuracy at 93.33%. These results suggest that ARTIFACTCOPILOT can reliably identify whether an

artifact is reviewable and whether its documentation provides a sufficient basis for evaluation.

Mismatch cases are concentrated more heavily on Functional and Reusable awards. In both cases, ARTIFACTCOPILOT shows higher precision than recall, indicating a conservative tendency in granting stronger positive decisions. This pattern is also reflected at the dimension level. D1 and D2 remain relatively stable, whereas D3, D4, and especially D5 are weaker, with D5 recording the lowest four-class accuracy. The gap between dimension-level and final badge-level agreement is expected because the final badge is a cumulative decision, an error on any gate-critical dimension can propagate to the overall badge outcome. Taken together, these results suggest that the main remaining difficulty lies not in basic repository inspection or setup recognition, but in turning execution evidence into stronger judgments about workflow validity, claim support, and especially reuse.

C. RQ2: Comparison with Baseline Agents

Table IV compares ARTIFACTCOPILOT with four baseline agents under three guidance conditions: ACM v1.1 badging policy [8], ARTIFACTGUIDE, and Skills+ARTIFACTGUIDE. Here, Skills+ARTIFACTGUIDE denotes the stage-specific phase skills from ARTIFACTCOPILOT combined with ARTIFACTGUIDE. This comparison is intended to test how progressively stronger review guidance affects agent behavior under our rubric-guided AE task formulation, rather than to reproduce the original venue badge process. At the badge level, ARTIFACTCOPILOT achieves 70.56% exact-match accuracy, exceeding the strongest baseline, Codex under Skills+ARTIFACTGUIDE, at 67.78%. At the Functional binary level, Codex under Skills+ARTIFACTGUIDE attains higher accuracy and recall, whereas ARTIFACTCOPILOT reaches a precision of 98.55%, the highest among all systems. This result indicates that ARTIFACTCOPILOT is less likely to translate partial execution success into unsupported Functional awards.

The protocol effect is clearest at the badge level. For OpenHands, Cline, and Codex, performance improves consistently as the guidance moves from ACM v1.1 to ARTIFACTGUIDE and then to Skills+ARTIFACTGUIDE. The gains are much larger for final badge prediction than for basic Functional

TABLE IV: RQ2: Comparison with baselines. Values are means across three runs; bold indicates the best result.

Agent	Protocol	Badge-level				Functional-level			
		Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
OpenHands	ACM v1.1	48.89%	48.18%	45.04%	43.10%	82.78%	84.56%	85.29%	84.78%
	ARTIFACTGUIDE	56.11%	53.88%	51.11%	49.62%	82.78%	83.24%	87.25%	85.15%
	Skills+ARTIFACTGUIDE	59.44%	64.74%	59.32%	61.38%	81.67%	83.65%	85.29%	84.44%
Cline	ACM v1.1	34.44%	37.41%	37.61%	33.91%	70.00%	76.61%	67.65%	71.84%
	ARTIFACTGUIDE	52.78%	54.92%	58.04%	52.47%	76.11%	80.49%	76.47%	78.35%
	Skills+ARTIFACTGUIDE	59.44%	67.50%	61.45%	64.05%	80.56%	83.33%	83.33%	83.40%
Codex	ACM v1.1	39.44%	41.81%	33.89%	34.76%	73.89%	71.65%	89.22%	79.44%
	ARTIFACTGUIDE	55.56%	66.65%	52.76%	57.69%	76.67%	78.85%	80.39%	79.63%
	Skills+ARTIFACTGUIDE	67.78%	71.92%	66.72%	68.71%	86.11%	85.32%	91.18%	88.13%
ExecutionAgent	ACM v1.1	31.11%	37.40%	30.83%	29.72%	56.11%	68.57%	41.18%	51.27%
	ARTIFACTGUIDE	33.89%	37.93%	36.52%	32.21%	65.56%	83.62%	50.00%	62.14%
	Skills+ARTIFACTGUIDE	32.78%	44.67%	41.71%	30.85%	58.33%	94.41%	28.43%	43.22%
ARTIFACTCOPILOT		70.56%	74.05%	72.05%	70.64%	82.78%	98.55%	70.59%	82.23%

TABLE V: Stability and cost across three runs. For each baseline, we report its best-performing protocol in RQ2. Cost is the mean API cost per artifact run.

System	Protocol	Badge consistency	Badge divergence	Functional agreement	Failure rate	Cost
OpenHands	Skills+ARTIFACTGUIDE	35/60	6/60	41/60	8.33%	\$0.043
Cline	Skills+ARTIFACTGUIDE	21/60	3/60	43/60	16.67%	\$0.068
Codex	Skills+ARTIFACTGUIDE	37/60	3/60	49/60	10.00%	\$0.044
ExecutionAgent	ARTIFACTGUIDE	12/60	15/60	37/60	45.00%	\$0.093
ARTIFACTCOPILOT	N/A	32/60	1/60	45/60	0.00%	\$0.125

discrimination, suggesting that the main value of the rubric and staged skills is not merely to help agents execute artifacts, but to help them organize evidence into more calibrated badge judgments. Trajectory analysis explains why Codex is the strongest baseline. Its largest advantage comes from D3 recovery behavior rather than from uniformly better judgment. In several mismatch cases, ARTIFACTCOPILOT follows one documented entry point, encounters a technical failure, and then commits to a lower score, whereas Codex continues exploring alternative documented paths and recovers executable evidence, which accounts for a substantial fraction of the gap between the two systems. The traces also explain why ARTIFACTCOPILOT still achieves the best overall badge-level result. Its phase constraints and packet checking make positive decisions more conservative, especially at the Functional gate. As a result, ARTIFACTCOPILOT produces fewer unsupported upgrades from partial execution evidence, even though this conservatism also causes it to miss some human-awarded positive cases. ExecutionAgent shows an opposite failure mode, that despite strong execution-oriented behavior, it does not reliably shift to evidence-centered review under ARTIFACTGUIDE or Skills+ARTIFACTGUIDE, indicating that execution ability alone is insufficient for AE.

Overall, RQ2 supports two conclusions. First, official badge descriptions alone do not provide enough structure for stable automated review. Both ARTIFACTGUIDE and

Skills+ARTIFACTGUIDE improve general-purpose agents substantially. Second, AE performance depends not only on whether an agent can run artifacts, but on whether it can turn execution into a bounded, evidence-driven judgment process.

D. RQ3: Stability Across Repeated Runs

We select the best-performing protocol configuration for each baseline from RQ2 as the basis for stability analysis. Table V reports stability across three independent runs under these selected configurations. Codex under Skills+ARTIFACTGUIDE achieves the highest exact three-run badge consistency at 37/60 and the highest Functional agreement at 49/60, indicating the strongest outcome convergence among runs that completed successfully. ARTIFACTCOPILOT is slightly lower on exact consistency at 32/60, but it is the only system with a zero failure rate, and it shows the fewest cases of extreme disagreement, with only 1 artifact receiving three different badge outcomes. Therefore, its advantage lies less in producing identical labels on every run than in avoiding runtime breakdowns and large result swings.

The sources of instability differ across baselines. OpenHands under Skills+ARTIFACTGUIDE produces more cases with three different badge outcomes across runs, suggesting that different runs often follow different review paths. Cline under Skills+ARTIFACTGUIDE shows lower exact consistency but limited extreme divergence, indicating more frequent

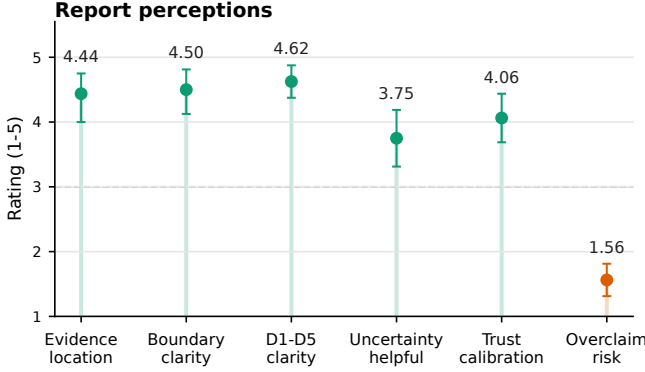


Fig. 3: Participant ratings of report qualities.

movement between adjacent badge levels. Its failure traces repeatedly show context overflow, a pattern also observed in Codex under Skills+ARTIFACTGUIDE. Codex benefits from shorter and more direct review trajectories, which helps explain its stronger consistency, but its failures still reveal sensitivity to long accumulated context and often appear as hard interruptions once the context budget is exceeded. We therefore interpret these failures not as artifact-specific difficulty, but as a context-management limitation arising from the interaction between long-horizon agent trajectories and the underlying model context bound. ExecutionAgent under ARTIFACTGUIDE presents a different failure mode. It has the lowest exact consistency at 12/60, the highest badge divergence at 15/60, and a failure rate of 45.00%. Trace inspection shows that these failures are dominated by workspace initialization conflicts, artifact clone failures, broken container working directories, corrupted screen sessions, and missing parseable badge outputs rather than by artifact-specific difficulty. This suggests that an agent optimized for project execution is not naturally suited to the long-horizon, staged, and auditable evidence review process required by AE.

Overall, RQ3 yields two findings. First, exact three-run consistency favors agents with shorter trajectories and more direct decisions, which explains the strength of Codex on this metric. Second, for artifact evaluation, the more important notion of stability is not only whether repeated runs return identical badges, but whether they complete reliably and keep result variation within a limited range. In this sense, ARTIFACTCOPILOT provides a more suitable stability profile for AE assistance.

E. RQ4: Reviewer Study

We conducted a controlled user study to examine how ARTIFACTCOPILOT reports support rubric-guided artifact evaluation. We selected 8 paper-artifact pairs from prior work in our lab and generated ARTIFACTCOPILOT reports for each and then prepared a with-report version and a no-report version. We recruited 8 participants with at least 3 years of software engineering research experience, none of whom had participated in the rubric design interviews. Following a

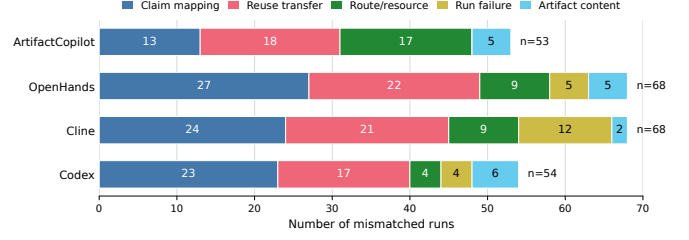


Fig. 4: Distribution of mismatch categories.

Latin-square design, each participant completed 4 review tasks (2 with-report and 2 no-report), with each artifact evaluated under both conditions by different reviewers. All tasks used ARTIFACTGUIDE-guided AE, requiring participants to record review outcomes and complete scoring forms. The study yielded 32 valid review records in total.

Because of the limited sample size, we treat the results as descriptive comparisons. Self-reported confidence and all report perception items were collected on five-point Likert scales from 1 (lowest) as the lowest rating to 5 (highest) as the highest [26]. Reviewers with access to the report reported higher confidence than those without it, with mean scores of 4.06 versus 3.63. Figure 3 summarizes participants’ assessments of the report itself. Evidence location received a mean score of 4.44 out of 5, boundary clarity 4.50, and dimension clarity 4.62. Uncertainty helpfulness and trust calibration were rated 3.75 and 4.06, while overclaim risk remained low at 1.56. These results suggest that the report was effective at helping reviewers locate evidence, understand what the current review did and did not establish, and follow the D1–D5 judgment structure. In the 16 with-report reviews, participant decisions agreed with the report in 15 cases. This suggests that the report was consistent with reviewer judgment in this controlled setting.

VI. DISCUSSION

A. Design Implications for Automated AE

Our mismatch analysis shows that the main barrier to automated artifact evaluation is no longer basic execution, but evidence interpretation. Across ARTIFACTCOPILOT, OpenHands, Cline, and Codex, many remaining errors arise after the artifact has already reached an executable state. We exclude ExecutionAgent from this analysis because most of its failures are infrastructure-level breakdowns before a usable AE judgment is produced, rather than interpretable review-decision mismatches. As Figure 4 shows, the dominant mismatch categories are claim mapping and reuse transfer, which together account for most disagreements. Both require evaluative judgments: whether an observed output is sufficiently connected to a paper claim, and whether a demonstrated use case constitutes genuine reuse rather than a minor workflow variation. This also exposes a limitation of our current system. Although ARTIFACTCOPILOT improves execution control and evidence organization, it still remains sensitive to execution context

when interpreting evidence. The same artifact may lead to different badge outcomes depending on route choice, resource availability, or whether a repair is treated as acceptable intervention. These errors are therefore not merely implementation failures, but symptoms of the underlying subjectivity and context dependence of AE itself.

Taken together, these findings suggest that automated AE should not be framed as autonomous badge issuance from opaque execution outcomes. Instead, it should be designed as a reviewer-oriented evidence assistant that makes route choices explicit, records intervention burden, exposes uncertainty in claim mapping and reuse judgments, and localizes where human adjudication is still needed.

B. Design Implications for Artifacts

Our findings also suggest that ARTIFACTCOPILOT can serve as a detector of artifact quality, not only an evaluator. Artifacts that can be assessed stably by ARTIFACTCOPILOT are typically those with clearer structure, more explicit evidence, and better-defined review paths. Conversely, when the agent becomes uncertain or unstable, the cause often lies not in the code alone, but in underspecified artifact design. In this sense, evaluation difficulty is itself a useful signal of artifact quality.

This observation points to several concrete design practices for artifacts. First, artifacts should expose a canonical evaluation route, including the intended entry point, required resources, and acceptable bounded substitutes when full execution is too costly. Second, they should make outputs legible as evidence by explicitly linking scripts, logs, or tables to the corresponding paper claims. Third, artifacts targeting Reusable should provide at least one bounded transfer case with expected output, so that reuse can be validated rather than inferred. Finally, external dependencies such as GPUs, model downloads, API keys, or large datasets should be stated explicitly, together with a reduced validation path whenever possible. A high-quality artifact should therefore not be treated as merely a public code release, but should be designed and organized according to the standards of high-quality research software. Whether or not an artifact is submitted to AE, it should make its review path, the connection between outputs and paper claims, and its reusable validation path as explicit as possible, because these properties improve not only the stability of automated evaluation but also the practicality of human review, reproduction, and downstream use.

C. Threats to Validity

The main internal threat lies in the construction of the adjudicated badges. Artifact evaluation inevitably involves judgment calls, especially when deciding whether an execution is representative, whether an output supports a paper claim, and whether a bounded transfer case establishes reuse. Because both the badges and ARTIFACTCOPILOT are grounded in the finalized ARTIFACTGUIDE, some degree of rubric-alignment bias cannot be fully excluded. We mitigate this threat through independent review, discussion, and third-party adjudication.

The main external threat lies in scope. Our 60-artifact suite was obtained by stratified random sampling to cover positive, negative, and difficult boundary cases, rather than to favor any system or estimate population frequency. The results also depend on the selected model backend, agent frameworks, and execution budgets. We therefore present ARTIFACTCOPILOT as a controlled instantiation, and view our findings as evidence about recurring challenges in AE automation rather than as universally generalizable performance results.

VII. RELATED WORK

A. Artifact Evaluation Practice and Standards

Prior work has examined different aspects of AE. Krishnamurthi et al. [21] discussed the early motivation and review process of AE at the conference level. Hermann et al. [17] analyzed reviewer expectations and showed that existing guidelines do not provide sufficiently uniform criteria for artifact judgment. Saucez et al. [31] highlighted the tension between badge assignment and deeper validation. Timperley et al. [33] studied recurring obstacles to reuse, including incomplete documentation and undeclared dependencies. These studies characterize AE practice, but they stop short of operationalizing badge policy into a usable scoring rubric. ARTIFACTGUIDE is designed to address this gap.

B. LLM Agents and Automated Evaluation

LLM agents combine reasoning, tool use, and iterative feedback [44], [37], [41]. In software engineering, they have been applied to repository navigation [43], [45], general-purpose code development [39], code repair [10], automated project execution [11], vulnerability detection [23], [22], and test generation [24], [38]. These systems are not targeting evidence-based reviews. In parallel, Pathak et al. showed that problem-specific rubrics significantly improve LLM code evaluation [28], and the agent-as-a-judge paradigm extended agents to executable dynamic evaluation [46], [16]. In AE, Heye et al. proposed an LLM-driven method for extracting reproducibility defects in cybersecurity artifacts [18], but their work focuses on reproducibility signals rather than full badge-oriented review. More recently, Baek and Pradel proposed Artisan for agentic artifact evaluation [9], focusing on validating paper-reported result objects such as tables against artifact outputs. Our work instead targets the complete AE workflow. ARTIFACTGUIDE and ARTIFACTCOPILOT support this workflow by combining a systematically constructed rubric with a phase-driven agent.

VIII. CONCLUSION

We introduced ARTIFACTCOPILOT and ARTIFACTGUIDE, arguing that AE automation is best understood as a rubric-guided evidence review task. ARTIFACTGUIDE translates descriptive badge standards into operational review criteria, while ARTIFACTCOPILOT realizes them through phase-specific review control. On 60 real artifacts with human-adjudicated badges, ARTIFACTGUIDE improves the AE performance of three coding agents over official policy prompts,

and ARTIFACTCOPILOT achieves the highest exact badge agreement at 70.56% while completing all repeated runs with zero failure rate. Our analysis shows that the main remaining challenge of automated AE is evidence interpretation rather than execution alone. It further suggests that higher-quality artifacts should make their review path, claim support, and reuse boundary explicit.

REFERENCES

- [1] “Artifactguide,” <https://figshare.com/s/4a21d79f93f92d11f2a1>, accessed: 2026-07-01.
- [2] “ASE 2026 artifact evaluation track,” <https://conf.researchr.org/track/ase-2026/ase-2026-artifact-evaluation>, 2026, 41st IEEE/ACM International Conference on Automated Software Engineering, 2026.
- [3] “FSE 2026 artifacts track,” <https://conf.researchr.org/track/fse-2026/fse-2026-artifacts>, 2026, the ACM International Conference on the Foundations of Software Engineering, 2026.
- [4] “ICSE 2026 artifact evaluation track,” <https://conf.researchr.org/track/icse-2026/icse-2026-artifact-evaluation>, 2026, 48th IEEE/ACM International Conference on Software Engineering, Rio de Janeiro, Brazil, April 12–18, 2026.
- [5] “ISSTA 2026 artifact evaluation track,” <https://conf.researchr.org/track/issta-2026/issta-2026-artifact-evaluation>, 2026, 35th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2026.
- [6] ACM SIGMOBILE, “ACM SIGMOBILE Research Papers Artifact Evaluation Guidelines,” <https://www.sigmobile.org/grav/about/artifact-guidelines>, n.d., prepared by the Research Artifact Evaluation Advisory Committee. Accessed: June 17, 2026.
- [7] Anthropic, “Agent skills,” Anthropic Platform Documentation, 2026, accessed: 2026-06-29. [Online]. Available: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- [8] Association for Computing Machinery, “Artifact review and badging – current,” <https://www.acm.org/publications/policies/artifact-review-and-badging-current>, Aug. 2020, version 1.1, August 24, 2020. Accessed: June 17, 2026.
- [9] D. Baek and M. Pradel, “Artisan: Agentic artifact evaluation,” *arXiv preprint arXiv:2602.10046*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.10046>
- [10] I. Bouzenia, P. Devanbu, and M. Pradel, “Repairagent: An autonomous, llm-based agent for program repair,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE ’25. IEEE Press, 2025, p. 2188–2200. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00157>
- [11] I. Bouzenia and M. Pradel, “You name it, i run it: An llm agent to execute tests of arbitrary projects,” *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3728922>
- [12] V. Braun and V. Clarke, “Using thematic analysis in psychology,” *Qualitative Research in Psychology*, vol. 3, no. 2, pp. 77–101, 2006. [Online]. Available: <https://doi.org/10.1191/1478088706qp0630a>
- [13] Cline Bot, “Cline bot: Ai coding assistant,” <https://cline.bot/>, 2024, accessed: 2026-06-23.
- [14] D. C. D’Elia, T. D. Doudali, C. Giuffrida, M. Matos, M. Payer, S. Pirelli, G. Portokalidis, V. Schiavoni, S. Signorello, and A. Vahldiek-Oberwagner, “Lessons learned from five years of artifact evaluations at eurosys,” in *Proceedings of the 3rd ACM Conference on Reproducibility and Replicability*, ser. ACM REP ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 108–120. [Online]. Available: <https://doi.org/10.1145/3736731.3746152>
- [15] Farion, “Cc-switch,” 2025, accessed 2026-06-29. [Online]. Available: <https://github.com/farion1231/cc-switch>
- [16] B. Gou, Z. Huang, Y. Ning, Y. Gu, M. Lin, W. Qi, A. Kopanav, B. Yu, B. J. Gutierrez, Y. Shu, C. H. Song, J. Wu, S. Chen, H. N. Moussa, T. ZHANG, J. Xie, Y. Li, T. Xue, Z. Liao, K. Zhang, B. Zheng, Z. Cai, V. Rozgic, M. Ziyadi, H. Sun, and Y. Su, “Mind2web 2: Evaluating agentic search with agent-as-a-judge,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. [Online]. Available: <https://openreview.net/forum?id=AUaW6DS9si>
- [17] B. Hermann, S. Winter, and J. Siegmund, “Community expectations for research artifacts and evaluation processes,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 469–480. [Online]. Available: <https://doi.org/10.1145/3368089.3409767>
- [18] D. Heye, K. Kindermann, R. Decker, J. Lohmöller, A. Belova, S. Geisler, K. Wehrle, and J. Pennekamp, “Supporting artifact evaluation with llms: A study with published security research papers,” in *2025 IEEE International Conference on Big Data (BigData)*, 2025, pp. 5077–5085. [Online]. Available: <https://doi.org/10.1109/BigData66926.2025.11401815>
- [19] A. Jonsson and G. Svingby, “The use of scoring rubrics: Reliability, validity and educational consequences,” *Educational Research Review*, vol. 2, no. 2, pp. 130–144, 2007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1747938X07000188>
- [20] S. Klein-Gardner, L. Abts, and G. Goldberg, “The engineering design process portfolio scoring rubric (edppsr) – initial validity and reliability (fundamental),” in *2022 ASEE Annual Conference & Exposition*, no. 10.18260/1-2-41236. Minneapolis, MN: ASEE Conferences, August 2022, <https://peer.asee.org/41236>. [Online]. Available: <https://peer.asee.org/41236>
- [21] S. Krishnamurthi, “Artifact evaluation for software conferences,” *SIGPLAN Not.*, vol. 48, no. 45, p. 17–21, Jul. 2013. [Online]. Available: <https://doi.org/10.1145/2502508.2502518>
- [22] F. Liu, Y. Zhang, J. Luo, J. Dai, T. Chen, L. Yuan, Z. Yu, Y. Shi, K. Li, C. Zhou, H. Chen, and M. Yang, “Make agent defeat agent: Automatic detection of Taint-Style vulnerabilities in LLM-based agents,” in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 3767–3786. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu>
- [23] J. Liu, W. Hao, K. Cheng, and D. Jin, “Large language model-based planning agent with generative memory strengthens performance in textualized world,” *Eng. Appl. Artif. Intell.*, vol. 148, p. 110319, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:276859598>
- [24] N. Mündler, M. N. Mueller, J. He, and M. Vechev, “Code agents are state of the art software testers,” in *ICML 2024 Workshop on Foundation Models in the Wild*, 2024. [Online]. Available: <https://openreview.net/forum?id=LfyYSHGQMZ>
- [25] A. Muttakin, S. Mondal, and C. K. Roy, “The state of open science in software engineering research: A case study of icse artifacts,” *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.02066>
- [26] T. Nemoto and D. Beglar, “Developing likert-scale questionnaires,” in *JALT2013 Conference Proceedings*, N. Sonda and A. Krause, Eds. Tokyo: JALT, 2014, pp. 1–8. [Online]. Available: https://jalt-publications.org/sites/default/files/pdf-article/jalt2013_001.pdf
- [27] OpenAI, “Codex,” <https://openai.com/codex/>, 2026, accessed: 2026-06-28.
- [28] A. Pathak, R. Gandhi, V. Uttam, A. Ramamoorthy, P. Ghosh, A. R. Jindal, S. Verma, A. Mittal, A. Ased, C. Khatri, Y. Nakka, Devansh, J. S. Challa, and D. Kumar, “Rubric is all you need: Improving llm-based code evaluation with question-specific rubrics,” in *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1*, ser. ICER ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 181–195. [Online]. Available: <https://doi.org/10.1145/3702652.3744220>
- [29] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. Maddison, and T. Hashimoto, “Identifying the risks of lm agents with an llm-emulated sandbox,” in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 27031–27098. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/7274ed909a312d4d869cc328ad1c5f04-Paper-Conference.pdf
- [30] J. Rudolph, F. Ismail, S. Tan, and P. Seah, “Don’t believe the hype. ai myths and the need for a critical approach in higher education,” pp. 6–27, 2025.
- [31] D. Saucez, L. Iannone, and O. Bonaventure, “Evaluating the artifacts of sigcomm papers,” *SIGCOMM Comput. Commun. Rev.*, vol. 49, no. 2, p. 44–47, May 2019. [Online]. Available: <https://doi.org/10.1145/3336937.3336944>

- [32] G. Sherman and S. Nagy, “No harness, no problem: Oracle-guided harnessing for auto-generating c api fuzzing harnesses,” in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE ’25. IEEE Press, 2025, p. 165–177. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00239>
- [33] C. S. Timperley, L. Herckis, C. Le Goues, and M. Hilton, “Understanding and improving artifact sharing in software engineering research,” *Empirical Software Engineering*, vol. 26, no. 4, p. 67, 2021. [Online]. Available: <https://doi.org/10.1007/s10664-021-09973-5>
- [34] C. Treude, C. M. Poskitt, and R. Hoda, “Rethinking artifact evaluation for software engineering in the age of generative ai,” *arXiv preprint arXiv:2604.16306*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.16306>
- [35] S. M. Vieira, U. Kaymak, and J. M. C. Sousa, “Cohen’s kappa coefficient as a performance measure for feature selection,” in *International Conference on Fuzzy Systems*, 2010, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/FUZZY.2010.5584447>
- [36] T. N. Vijaykumar, “A common standard to fix our review process,” *SIGARCH Blog*, apr 2018. [Online]. Available: <https://www.sigarch.org/a-common-standard-to-fix-our-review-process-and-oh-i-was-wrong-about-one-thing/>
- [37] L. Wang, C. Ma, X. Feng, Z. Zhang, H. ran Yang, J. Zhang, Z.-Y. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. rong Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261064713>
- [38] X. Wang, P. Gao, X. Meng, C. Peng, R. Hu, Y. Lin, and C. Gao, “Aegis: An agent-based framework for bug reproduction from issue descriptions,” in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 331–342. [Online]. Available: <https://doi.org/10.1145/3696630.3728557>
- [39] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, “Openhands: An open platform for AI software developers as generalist agents,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=OJd3ayDDoF>
- [40] S. Winter, C. S. Timperley, B. Hermann, J. Cito, J. Bell, M. Hilton, and D. Beyer, “A retrospective study of one decade of artifact evaluations,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 145–156. [Online]. Available: <https://doi.org/10.1145/3540250.3549172>
- [41] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, “The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025. [Online]. Available: <https://doi.org/10.1007/s11432-024-4222-0>
- [42] W. Xu, C. Huang, S. Gao, and S. Shang, “LLM-Based Agents for Tool Learning: A Survey,” *Data Science and Engineering*, pp. 1–31, 2025. [Online]. Available: <https://doi.org/10.1007/s41019-025-00296-9>
- [43] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: agent-computer interfaces enable automated software engineering,” in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS ’24. Red Hook, NY, USA: Curran Associates Inc., 2024. [Online]. Available: <https://dl.acm.org/doi/10.5555/3737916.3739517>
- [44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [45] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: ACM, 2024, pp. 1592–1604. [Online]. Available: <https://doi.org/10.1145/3650212.3680384>
- [46] M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, “Agent-as-a-judge: Evaluate agents with agents,” in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=Nn9POI9Ekt>